VM Console — Vulnerability Management Platform

Project & DevOps Documentation

---

1. Objective

Build and productionize **VM Console** — a self-hosted vulnerability management platform that lets a security team run a suite of open-source scanners against their code, dependencies, containers, and web applications from one console, track findings through a full lifecycle, and manage access across tenants.

The primary engineering objective — and the focus of this document — was standing up a real, working **CI/CD pipeline**: GitHub → GitHub Actions → Docker → AWS, for both a NestJS backend and a Next.js frontend, built entirely on AWS free-tier infrastructure, with every step actually verified running rather than just configured.

---

2. How It Works — System Overview

Two independently containerized, independently deployed services:

- **Frontend** (Next.js 14, App Router) — the console UI.
- **Backend** (NestJS) — API, auth/RBAC, and the scanner orchestration engine.

The backend connects to **MongoDB Atlas** (cloud, free tier) for persistence, and to its own host's Docker daemon (via a mounted `docker.sock`) to spawn containerized scanners like OWASP ZAP at scan time.

Diagram source (Mermaid) -- renders visually in the Markdown/GitHub version of this document

```
flowchart LR
    User[Security Analyst] -->|HTTPS| FE[Frontend<br/>Next.js — EC2 :3101]
    FE -->|REST API| BE[Backend<br/>NestJS — EC2 :3102]
    BE -->|Mongoose| DB[(MongoDB Atlas<br/>Free Tier)]
    BE -->|docker.sock| Scanners[Scanner Toolchain<br/>Semgrep, Gitleaks,
OSV,<br/>Trivy, Dependency-Check,<br/>Nmap, ZAP]
    Scanners -->|Findings| BE
```

---

3. Purpose of Vulnerability Management

A vulnerability management program continuously answers four questions:

1. **What do we have?** — asset/repository inventory.
2. **What's wrong with it?** — automated scanning across SAST, SCA, secrets, IaC misconfiguration, container images, and exposed network services.
3. **How bad is it, and who owns it?** — triage, assignment, risk scoring, formal exceptions with approval.
4. **Did we actually fix it?** — status lifecycle and evidence, so remediation is auditable, not just claimed.

This platform exists to make that loop run on every code change rather than as an occasional manual audit — which is exactly why the CI/CD discipline documented below matters: the same automation applied to *building* this platform is what it's meant to bring to the *code it scans*.

4. Tools & Technologies

Application

Layer	Technology
Backend	NestJS (Node.js)
Frontend	Next.js 14, Tailwind CSS
Database	MongoDB Atlas
Auth	JWT (access + refresh), bcrypt

Scanner toolchain

Semgrep (SAST) · Gitleaks (secrets) · OSV-Scanner & npm audit (SCA) · Trivy (filesystem/IaC/image) · OWASP Dependency-Check (SCA, Java-focused) · Nmap (network) · OWASP ZAP (DAST, spawned as a sibling container)

Checkov, Prowler, ScoutSuite were removed — confirmed unused by any code path and the largest contributors to image size.

DevOps & infrastructure

Git/GitHub · GitHub Actions · self-hosted runner (WSL2) · Docker (multi-stage builds) · Docker Hub · AWS EC2 (free tier) · MongoDB Atlas (free tier)

5. Workflow Diagram

Diagram source (Mermaid) -- renders visually in the Markdown/GitHub version of this document

```
flowchart TD
    Dev[Developer] -->|git push| Repo[GitHub Repository]
    Repo --> Actions[GitHub Actions Triggered]
    Actions --> Test[Job: Build & Lint<br/>GitHub-hosted runner]
    Test -->|pass| Build[Job: Build & Push<br/>Self-hosted runner]
    Build --> DockerBuild[docker build<br/>local image]
    DockerBuild --> Push[docker push → Docker Hub]
    Push --> Deploy[Job: Deploy<br/>Self-hosted runner]
    Deploy -->|SSH| EC2[AWS EC2 Instance]
    EC2 --> Pull[docker pull latest]
    Pull --> Restart[stop/rm old → run new]
    Restart --> Live[🌐 Live Application]
    Test -->|fail| Stop1[Pipeline stops]
    DockerBuild -->|fail| Stop2[Pipeline stops]
```

6. How It Works — Runtime Behavior

On a user request, the frontend calls the backend's REST API at its EC2 public IP. The backend authenticates via JWT, resolves the tenant/RBAC context, and either serves data from MongoDB directly or, for a scan request, spawns the relevant scanner tool — either a locally-installed binary/venv inside the backend container, or (for ZAP) a fresh Docker container on the same host, using the mounted `docker.sock`. Scanner output is parsed into a normalized finding schema and written back to MongoDB, where it becomes visible in the console immediately.

7. DevOps Deep Dive — CI/CD, Docker & AWS

This is the core of the project. Everything below was built, broken, debugged, and re-verified running — not just written once and assumed correct.

7.1 Pipeline architecture

Each repository (`VM_frontend`, `VM_backend`) runs an independent three-job `deploy.yml`:

Diagram source (Mermaid) -- renders visually in the Markdown/GitHub version of this document

```
flowchart LR
    subgraph Job1["test (GitHub-hosted)"]
        A1[npm ci] --> A2[npm run build] --> A3[npm run lint]
    end
    subgraph Job2["build-and-push (self-hosted)"]
        B1[docker build] --> B2[docker login] --> B3[docker push]
    end
    subgraph Job3["deploy (self-hosted)"]
        C1[ssh to EC2] --> C2[docker pull] --> C3[stop/rm old container] --> C4[docker
run new container]
    end
    Job1 --> Job2 --> Job3
```

- **`test`** runs on a standard GitHub-hosted runner — cheap, disposable, no reason to occupy the self-hosted machine for lint/build checks.
- **`build-and-push`** and **`deploy`** run on a **self-hosted runner**, registered at the **organization level** (not per-repo) so one runner serves both `VM_frontend` and `VM_backend` without duplicate registrations.

7.2 Why self-hosted, and the real trade-off

The build step runs on the developer's own WSL2 machine so the built image is visible directly in Docker Desktop during development, and so no cloud build-minute or registry cost applies. The trade-off, made explicit rather than discovered later: **the pipeline is only as "automatic" as the runner machine being on.** A push made while the laptop is asleep queues with no runner to pick it up until it's back.

Security implication also documented up front: self-hosted runners execute arbitrary workflow code on that machine. The workflow triggers on `push` only — never `pull_request` — specifically to avoid a fork PR running code on the developer's own hardware.

7.3 Docker — frontend

Standard two-stage Next.js build: `build` stage (`npm ci`, `npm run build`, `output: "standalone"`) → slim `runtime` stage copying only the standalone output, non-root `appuser`, container `HEALTHCHECK` against `/login`.

7.4 Docker — backend (the harder image)

Multi-stage build bundling an entire scanner toolchain into one runtime image:

- System layer: Python 3 + venv, OpenJDK 17 headless (Dependency-Check), Nmap, Docker CLI **without a daemon** (talks to the host's daemon over the mounted socket).
- Go/binary scanners (Gitleaks, OSV-Scanner, Trivy, Dependency-Check) downloaded at build time from **version-pinned** GitHub Releases — pins had to be actively maintained during this project when an old Trivy pin's release assets were pruned, and when the Dependency-Check upstream repo was archived and moved orgs.
- Only Semgrep (the one Python scanner the app actually calls) gets a venv — Checkov/Prowler/ScoutSuite were removed after confirming via `grep` against the app's own scanner registry that no code path ever

invokes them; they were also the single largest image-size contributor (each pulls in full AWS/Azure/Alibaba Cloud SDKs).

- Runs as non-root `appuser`, added to a `dockerhost` group whose GID is passed in as `--build-arg DOCKER_HOST_GID`. **This must match the real host's Docker group GID** — confirmed on the actual EC2 instance via `getent group docker` (109, not the Dockerfile's default of 999) before the socket mount would actually grant usable access.

7.5 The `docker.sock` trade-off

Bind-mounting `/var/run/docker.sock` into the backend container is required so it can spawn ZAP at scan time — and it is a documented, accepted security trade-off, not an oversight:

> *A container with access to the host's Docker socket has effective root on that host. Accepted at this project's scale; a hardened version would isolate scanner execution into its own sandboxed task instead of sharing the host daemon.*

7.6 AWS infrastructure — free tier by design

Component	Choice	Why
Compute	2x EC2 t2.micro/t3.micro, one per service	Free-tier eligible; two boxes avoid RAM contention between the lightweight frontend and the heavier scanner-laden backend
Storage	Default EBS volumes	Kept within the shared 30GB/month free-tier ceiling rather than resized, which forced the backend image-size reduction in 7.4
Database	MongoDB Atlas M0	Removes an entire in-process MongoDB container's RAM footprint from the free-tier instance
Registry	Docker Hub, public repos	ECR's 500MB free-tier storage limit would have been exceeded immediately by the multi-tool backend image
Networking	Public IP + security groups (SSH + app port only)	No ALB, no NAT gateway — both carry real ongoing cost even inside "free tier," and neither is needed at this scale

No managed orchestration (ECS/EKS) — each service is a single long-lived container per instance, replaced directly by the deploy job. This is the single biggest scope trade-off in the whole project: it fits entirely inside free tier, at the cost of every high-availability property a real production deployment would want.

7.7 Configuration & secrets

- **GitHub encrypted secrets**: Docker Hub token, SSH deploy key path.
- **GitHub variables**: EC2 IPs, SSH usernames, Docker Hub username (non-sensitive, log-visible).
- **Runtime config**: a `.env` file on each EC2 host, injected via `docker run --env-file` — never baked into the image, so the same image can be re-pointed at different environments without a rebuild.
- Two separate SSH keypairs (frontend, backend) were deliberately generated so a leaked key for one service doesn't grant access to the other.

7.8 Debugging log — real failures hit and fixed during this build

Documented because these are exactly the class of problem this pipeline will hit again:

Failure	Root cause	Fix
npm ci ERESOLVE conflict	@adminjs/mongoose required mongoose>=8; pinned to ^7.5.0	Removed the entire unused AdminJS/TypeORM/BullMQ/Redis dependency tree (confirmed zero usage)
Build failed: missing module	src/s3.service.ts referenced but absent from source	Implemented a local-disk-backed replacement matching the expected interface
Container crash-loop: MODULE_NOT_FOUND	A dev-only dependency (mongodb-memory-server) was statically imported, so it was required at process start even in production, after npm prune --omit=dev removed it	Converted to a dynamic import(), scoped inside the dev-only code branch
Docker build: Trivy download 404	Old pinned release's assets had been pruned upstream	Bumped to current release, verified live via direct HTTP check before committing
Docker build: Dependency-Check download failing	Upstream repo archived and moved to a new GitHub org	Repointed the download URL, verified live
docker pull: no space left on device	Backend image (multi-language scanner toolchain) exceeded the EC2 instance's default disk	Removed unused scanners rather than resizing, cutting image size and build time substantially
Login failing: "Token generation failed"	Code required JWT_ACCESS_SECRET / JWT_REFRESH_SECRET; env file only had a single unused JWT_SECRET	Added both correct variable names with independently generated random values

---

8. Pipeline Workflow — Trigger to Live

5. Developer pushes to the tracked branch.
6. GitHub Actions triggers `deploy.yml`.
7. **Test** job builds and lints on a GitHub-hosted runner — halts the pipeline on failure.
8. **Build** job (self-hosted runner) builds the Docker image from current source, tagged with the commit SHA.
9. Image pushed to Docker Hub under both the SHA tag and `latest`.
10. **Deploy** job SSHes into the target EC2 instance with a dedicated deploy key.
11. On the instance: pull new image -> stop/remove old container -> start new one with the same port mapping, restart policy, and (backend only) the same docker.sock mount and env file.
12. A container HEALTHCHECK confirms the app actually responds before Docker reports it healthy — the deploy job succeeding in GitHub's UI is not itself proof of a working app, and this project verified that distinction the hard way more than once.

---

9. Future Scope

Everything below is a deliberate, documented gap — not something overlooked — that would be the next investment if this moved beyond its current scale.

Pipeline hardening

- **Wire the existing test suite into CI.** Real Jest/e2e tests already exist in the backend repo but aren't run by deploy.yml — currently a build+lint gate only, no behavioral regression coverage before deploy.
- **Add a pre-push image scan.** Trivy is already part of the backend's own scanner toolchain; running it against the built image before docker push (failing on HIGH/CRITICAL) would close the irony of an unscanned vulnerability-management platform.
- **Manual approval gate before production deploys**, via a GitHub Environment with required reviewers — currently every push to main deploys immediately and unattended.
- **Rollback on failed health check.** Right now a deploy that starts but is functionally broken has no automatic revert path.

Infrastructure

- **Move off single-instance Docker to managed orchestration** (ECS or a lightweight Kubernetes) once free-tier constraints aren't the binding limit — enables rolling deploys, autoscaling, and eliminates the current single-container-per-host single point of failure.
- **Add an Application Load Balancer with TLS** and a real domain — both services are currently plain HTTP on raw IPs, fine for a demo, not for anything handling real credentials.
- **Persistent storage for evidence uploads.** Currently written to the backend container's local disk, which is wiped on every redeploy — needs a mounted volume or object storage (S3, or a free-tier equivalent) before this is usable beyond testing.
- **Infrastructure-as-code** (Terraform/Pulumi) for the EC2 instances, security groups, and DNS — currently provisioned manually through the AWS console, which is exactly the kind of undocumented drift this project's CI/CD work was meant to eliminate on the application side.

Operational maturity

- **Centralized logging and monitoring** — currently docker logs on each box is the only visibility; CloudWatch or a lightweight self-hosted stack would surface failures without requiring an SSH session to notice them.
- **Secrets manager** instead of plaintext .env files on each host — AWS Secrets Manager or Parameter Store, injected at container start rather than sitting on disk.
- **Runner resilience** — either a second self-hosted runner for redundancy, or a documented decision to move build-and-push back to GitHub-hosted runners once the "see it in Docker Desktop" requirement is no longer a priority, removing the single-machine dependency entirely.

Product-side scanner expansion

- **Reintroduce cloud-account scanning (Prowler/ScoutSuite)** as a separate, on-demand task or microservice — not bundled into the always-running backend image — if scanning live AWS/Azure/GCP accounts becomes an actual product requirement, wired to a real code path rather than shipped speculatively.
- **IaC scanning breadth** beyond Trivy's current coverage (Checkov reintroduced the same way, if needed) once there's a concrete use case driving it.

Document reflects the actual deployed and verified state of the VM Console platform as of this writing — frontend and backend both live, backend confirmed connected to MongoDB Atlas, authentication confirmed working end to end.